

Trabajo Práctico

Super Elizabeth Sis.



Materia: Programación 1, primer semestre 2026

Integrantes		
👤 Nombre	✉ Correo electrónico	🆔 DNI
Noelía Barrionuevo	nbarrionuevob26@gmail.com	37.038.375
Miguel Angel Luna Lobos	lunalobosmiguel@gmail.com	36.849.993

Introducción

En el siguiente trabajo práctico se nos presentó el desafío de desarrollar un videojuego cuyas características fueran similares a Mario Bros. Con la salvedad de que los roles se invierten y ahora la heroína es la princesa, llamada Elizabeth, y el rehén es Mario. Nuestra princesa tendrá como misión rescatar a Mario, que se encuentra cautivo en el castillo del malvado Rey Camin. Para llegar hasta el Rey, Elizabeth, debe enfrentarse con enemigos que le irán robando vidas si no logra vencerlos. Para el buen desarrollo de este video juego se deberá cumplir con los siguientes requisitos obligatorios :

- 1) La princesa Elizabeth debe aparecer en el medio de la pantalla
- 2) Islas flotantes por donde se desplace la princesa (algunas fijas y otras aleatorias)
- 3) Desplazamiento hacia la derecha e izquierda sin sobrepasar los límites de la pantalla.
- 4) Salto y caída. La princesa debe poder saltar hasta una cierta altura, caer en caída libre y además rebotar al tocar la parte inferior de las islas .
- 5) Enemigos voladores que aparezcan de ambos lados de la pantalla (izquierdo y derecho) con el fin de dañar a la princesa y quitarle vidas.
- 6) La princesa podrá realizar disparos para defenderse y matar a sus enemigos.

El juego presenta victoria cuando la Princesa llega al castillo, el cual se encontrará en el borde derecho de nuestra pantalla. Con la victoria lograda tendremos la finalización del juego. En caso contrario tendremos condición de derrota; esta se cumple cuando la princesa pierda todas sus vidas, las cuales serán 10.

Estructura del proyecto

El proyecto presenta una estructura base desde la cual se comenzó el desarrollo. Esta estructura incluye al paquete entorno.jar que contiene a la clase Entorno y también incluye la carpeta src dentro de la cual está el código del proyecto. Dentro de src existe una carpeta llamada juego que cuenta con una clase Juego, que es la clase principal del proyecto y que implementa la interfaz especificada en entorno.jar, InterfaceJuego.

La clase Entorno se ocupa de los gráficos. Controla el ciclo principal del Juego, llamando al método `tick()` del objeto de tipo InterfaceJuego que se le pasa como argumento al construir una instancia. Además, detecta teclas presionadas, movimientos del mouse y otros.

A continuación se realizará una breve descripción del flujo del programa para luego entrar en detalles sobre las clases implementadas, variables de instancias y la descripción de cada método.

Al ejecutarse la clase Juego se produce el siguiente flujo de forma ordenada:

- 1) Se ejecuta el método `main`.
- 2) Se crea una instancia de la clase Juego.
- 3) Dentro del constructor de Juego se crea un objeto de la clase Entorno.
- 4) El objeto entorno crea la ventana gráfica.
- 5) El objeto entorno inicia el ciclo principal del juego.
- 6) Durante el ciclo, el objeto entorno invoca al método `tick()`.
- 7) El método `tick()` continúa ejecutándose de forma repetitiva mientras la ventana del juego continúe abierta.

Clase Rectángulo

Las instancias de Rectángulo representan un rectángulo centrado en un punto (x, y) con un ancho y un alto determinados. Es la estructura geométrica base del juego: todos los elementos con colisión utilizan esta clase. Es inmutable (todos sus campos son final). Es utilizada por prácticamente todas las clases del juego como caja de colisión y como argumento en métodos de Rectangulos, Mundo, Isla, Princesa, Enemigo, Jefe, ProyectilPrincesa y Castillo.

Propiedades de instancia

- **x**: coordenada x del centro del rectángulo
- **y**: coordenada y del centro del rectángulo
- **ancho**: ancho del rectángulo
- **alto**: alto del rectángulo

Métodos de instancia

- **x()**: devuelve la coordenada x del rectángulo
- **y()**: devuelve la coordenada y del centro del rectángulo
- **ancho()**: devuelve el ancho del rectángulo
- **alto()**: devuelve el alto del rectángulo
- **bordeSuperior()**: devuelve $y - \text{alto}/2$ (borde superior)
- **bordeInferior()**: devuelve $y + \text{alto}/2$ (borde inferior)
- **bordeDerecho()**: devuelve $x + \text{ancho}/2$ (borde derecho)
- **bordeIzquierdo()**: devuelve $x - \text{ancho}/2$ (borde izquierdo)
- **dibujarRectangulo(Entorno entorno, Color color)**: Dibuja el rectángulo en pantalla con el color indicado
- **escalar(double ancho, double alto)**: devuelve un nuevo rectángulo con ancho y alto escalados
- **area()**: devuelve el área del rectángulo ($\text{ancho} * \text{alto}$)

Clase Fondo

Representa el fondo visual del juego. Es una imagen estática que se dibuja siempre en la misma posición de pantalla, independientemente del movimiento de la cámara.

Propiedades de instancia

- **x**: coordenada x
- **y**: coordenada y
- **fondo**: imagen de fondo

Métodos de instancia

- **dibujar(Entorno entorno)**: dibuja el fondo en su posición fija

Clase Isla

La clase Isla modela una plataforma flotante del escenario. Almacena su posición, tamaño e image. Además se encarga de dibujar y de informar a otros objetos del juego cuando ocurre una colisión con ella, detectando de qué lado se produce el choque y envía un mensaje al objeto involucrado, para que éste pueda reaccionar de forma adecuada. Por ejemplo informa a la princesa cuando se encuentra sobre tierra firme o al proyectil cuando debe rebotar. Para facilitar estas detecciones, la isla puede generar un Rectángulo que representa el área que ocupa dentro del mundo. Este rectángulo es utilizado por el sistema de colisión para determinar las interacciones con otras identidades.

Propiedades estáticas

- **proporcionAlto**: fracción de la altura del mundo que es ocupada por islas
- **proporcionAlturaMinima**: fracción de la altura del mundo que es la altura mínima de las islas
- **altoIsla**: altura de todas las islas
- **anchoMínimo**: ancho mínimo de las islas de nivel alto
- **anchoMaximo**: ancho máximo de las islas de nivel alto
- **factorFronteraAncho**: factor de expansión horizontal para detectar solapamiento con otras islas
- **factorFronteraAlto**: factor de expansión vertical para detectar solapamiento con otras islas
- **niveles**: número de niveles de islas
- **proporcionNivelesAltos**: proporcion del total de la altura que las islas ocupan que corresponde con niveles altos
- **anchoIslaNivelBajo**: el ancho fijo de las islas de nivel bajo
- **proporcionIslasBajas**: densidad de islas bajas respecto al área del mundo
- **proporcionIslasAltas**: densidad de islas altas respecto al área del mundo

Métodos estáticos

- **decimalRandom(double min, double max)**: Devuelve un double aleatorio en el intervalo [min, max).
- **crearImagenIsla(double ancho)**: devuelve un objeto java.awt.Image con la imagen de una isla del ancho proporcionado
- **nuevaNivelBajo(Mundo mundo)**: genera una isla de nivel bajo sin solapamientos
- **nuevaNivelAlto(Mundo mundo)**: genera una isla de nivel alto sin solapamientos
- **islaAleatoriaNivelBajo(Mundo mundo)**: genera una isla de nivel bajo sin verificar solapamiento
- **islaAleatoriaNivelAlto(Mundo mundo)**: genera una isla de nivel alto sin verificar solapamiento

Propiedades de instancia

- **x**: coordenada x del centro de la isla

- **y**: coordenada y del centro de la isla
- **ancho**: ancho de la isla
- **alto**: alto de la isla
- **isla**: imagen de la isla

Métodos de instancia

- **x()**: devuelve la coordenada x del centro de la isla
- **y()**: devuelve la coordenada y del centro de la isla
- **dibujar(Entorno entorno, Princesa princesa)**: dibuja la isla transformando sus coordenadas
- **actuarSobrePrincesa(Princesa princesa)**: detecta el tipo de colisión y envía el mensaje correspondiente a la princesa.
- **actuarSobreProyectilPrincesa(ProyectilPrincesa proyectilPrincesa)**: detecta el tipo de colisión y ordena al proyectil rebotar en la dirección correcta.

Clase Castillo

Representa el castillo al final del nivel. La princesa debe alcanzarlo para ganar. Es el objeto objetivo del juego. Su rectángulo de colisión usa solo el 20% del ancho total para requerir que la princesa esté bien posicionada al tocarlo.

Propiedades de instancia

- **x**: coordenada x del centro del castillo
- **y**: coordenada y del centro del castillo
- **ancho**: ancho del castillo
- **alto**: alto del castillo
- **castillo**: la imagen del castillo

Métodos de instancia

- **dibujar(Entorno entorno, Princesa princesa)**: dibuja el castillo aplicando una transformación de coordenadas
- **trasladar(double x, double y)**: cambia la posición del castillo
- **alto()**: devuelve el alto del castillo
- **rectángulo()**: devuelve el rectángulo de colisión del castillo

Clase Enemigo

Representa a un enemigo ordinario del juego. Se mueve horizontalmente con velocidad constante. Tiene un identificador único para poder ser encontrado eficientemente en el arreglo ordenado de Mundo. Al recibir el mensaje "morir" se marca para ser eliminado. Las instancias se crean en Enemigos, se almacenan en Mundo en un arreglo ordenado por ID. Juego lo dibuja, mueve y gestiona sus colisiones con la princesa y los proyectiles.

Propiedades estáticas

- **altoEnemigo**: alto de todos enemigos
- **anchoEnemigo**: ancho de todos los enemigos
- **minimoEnemigos**: mínimo de enemigos activos en pantalla
- **imagenEnemigo**: la imagen java.awt.Image que usan todos los enemigos

Métodos estáticos

- **nuevoEnemigoDerecha(int id, Princesa princesa, Mundo mundo, Entorno entorno, Juego juego)**: crea un enemigo que entra por el borde derecho y avanza a la izquierda
- **nuevoEnemigoIzquierda(int id, Princesa princesa, Mundo mundo, Entorno entorno, Juego juego)**: crea un enemigo que entra por el borde izquierdo y avanza a la derecha
- **enemigoAleatorio(int id, Mundo mundo, double x, double angulo)**: crea un enemigo en la posición dada con la dirección indicada
- **alturaAleatoria(int n, double yMin, double yMax)**: calcula la coordenada y correspondiente a un nivel de isla dado

Propiedades de instancia

- **id**: entero que hace las veces de identificador único
- **x**: coordenada x del centro del enemigo
- **y**: coordenada y del centro del enemigo
- **ancho**: ancho del enemigo
- **alto**: alto del enemigo
- **angulo**: ángulo de movimiento
- **velocidad**: velocidad de desplazamiento
- **vivo**: booleano que indica si el enemigo sigue vivo
- **imagen**: imagen del enemigo

Métodos de instancia

- **id()**: devuelve el identificador único del enemigo
- **dibujar(Entorno entorno, Princesa princesa)**: dibuja al enemigo transformado las coordenadas
- **mover()**: desplaza al enemigo según su ángulo y velocidad
- **debeEliminarse()**: devuelve true si el enemigo está muerto
- **rectangulo()**: devuelve el rectángulo de colisión
- **morir()**: marca al enemigo como muerto, para ser eliminado en el proximo tick.

Clase ProyectoilPrincesa

Representa el proyectil que dispara la princesa al hacer clic izquierdo con el mouse. Se mueve en línea recta hacia el cursor al momento del disparo. Rebota en las islas invirtiendo la componente de velocidad correspondiente al eje de la colisión.

Propiedades de instancia

- **x**: coordenada x del centro del proyectil
- **y**: coordenada y del centro del proyectil
- **ancho**: ancho del proyectil
- **alto**: alto del proyectil
- **cos**: componente horizontal del vector de dirección unitario
- **sen**: componente vertical del vector de dirección unitario
- **velocidad**: módulo de la velocidad del proyectil
- **imagen**: imagen del proyectil

Métodos de instancia

- **dibujar(Entorno entorno, Princesa princesa)**: dibuja el proyectil aplicando la transformación de coordenadas
- **mover()**: actualiza la posición del proyectil sumando $velocidad * cos$ y $velocidad * sen$ a x e y respectivamente
- **recibirMensaje(String mensaje)**: acepta "rebotar" lo cual invierte sen o cos según la dirección de colisión
- **rectangulo()**: devuelve el rectángulo de colisión del proyectil

Clase Princesa

Es el personaje principal del juego. Se mueve horizontalmente con las teclas de dirección, salta con la tecla A o flecha arriba, y dispara proyectiles hacia el cursor al hacer clic izquierdo. Implementa simulación de gravedad y caída libre con marcas temporales. Tiene un sistema de vidas representado por corazones en pantalla.

Propiedades estáticas

- **altoPrincesa**: alto configurado de la princesa
- **anchoPrincesa**: ancho configurado de la princesa
- **ladoCorazon**: el lado del cuadrado de cada corazón

Propiedades de instancia

- **x**: la coordenada x del centro de la princesa
- **y**: la coordenada y del centro de la princesa
- **ancho**: ancho de la princesa
- **alto**: alto de la princesa
- **velocidad**: velocidad de movimiento lateral
- **velocidadSalto**: velocidad vertical inicial al saltar
- **aceleracionGravitatoria**: aceleración de gravedad
- **velocidadCaídaLibre**: velocidad vertical acumulada en caída
- **enSalto**: indica si la princesa está ejecutando un salto
- **xNoCrece**: booleano que indica si la princesa se puede mover hacia la derecha

- **xNoDecrece**: booleano que indica si la princesa se puede mover hacia la izquierda
- **vidas**: vidas restantes
- **princesa**: imagen de la princesa
- **corazon**: imagen de un corazón usado para ilustrar las vidas de la princesa

Métodos de instancia

- **x()**: devuelve la coordenada x del centro de la princesa
- **y()**: devuelve la coordenada y del centro de la princesa
- **ancho()**: devuelve el ancho de la princesa
- **alto()**: devuelve el alto de la princesa
- **vidas()**: devuelve las vidas restantes de la princesa
- **trasladar(double x, double y)**: teletransporta a la princesa hacia las coordenadas del punto (x,y)
- **dibujar(Entorno entorno)**: dibuja a la princesa en el centro de la pantalla y dibuja los corazones que indican la cantidad de vidas restantes en la esquina superior izquierda de la pantalla
- **mover(Entorno entorno)**: aplica el movimiento lateral, salto y gravedad según el estado de las teclas y de la princesa
- **disparar(Entorno entorno, Juego juego)**: calcula el vector hacia el cursor y crea un `ProyectilPrincesa`
- **rectangulo()**: devuelve el rectángulo de colisión de la princesa
- **enTierraFirme()**: deja de aplicar la gravedad
- **cayendo()**: método interno que habilita la gravedad para el proximo tick
- **chocarMuroPorDerecha()**: no permite que la princesa se mueva hacia la derecha
- **chocarMuroPorIzquierda()**: no permite que la princesa se mueva hacia la izquierda
- **morir()**: deja a la princesa sin vidas
- **pierdeUnaVida()**: resta una vida de la princesa
- **gravedad()**: método interno específico para la dinámica de la gravedad
- **movimiento(double angulo, double velocidad)**: método interno específico de la dinámica del movimiento, solo verifica choques laterales, por eso es de uso interno.
- **movimientoLateral()**: método interno que regula la velocidad del movimiento lateral y lo ejecuta llamando al método `movimiento`.

Clase Jefe

Representa al enemigo jefe del nivel. Patrulla de lado a lado sobre una isla específica, cambiando de dirección al llegar a los bordes. Tiene 10 vidas y una barra de salud visible en pantalla. Matar al jefe (disparándole con el proyectil) no es condición de victoria, pero es el obstáculo más peligroso: tocarlo mata a la princesa instantáneamente.

Propiedades estáticas

- **altoJefe**: el alto configurado del jefe
- **anchoJefe**: el ancho configurado del jefe

Propiedades de instancia

- **x**: la coordenada x del centro del jefe
- **y**: la coordenada y del centro del jefe
- **ancho**: el ancho del jefe
- **alto**: el alto del jefe
- **cos**: componente horizontal del vector de dirección
- **velocidad**: el módulo de la velocidad
- **vidas**: las vidas restantes del jefe
- **isla**: isla sobre la cual patrulla el jefe
- **jefeHaciaDerecha**: imagen del jefe cuando avanza hacia la derecha
- **jefeHaciaIzquierda**: imagen del jefe cuando avanza hacia la izquierda
- **jefe**: imagen actual del jefe

Métodos de instancia

- **establecerIsla(Isla isla)**: coloca al jefe sobre la isla indicada
- **x()**: devuelve la coordenada x del jefe
- **y()**: devuelve la coordenada y del jefe
- **ancho()**: devuelve el ancho del jefe
- **alto()**: devuelve el alto del jefe
- **vidas()**: devuelve las vidas restantes del jefe
- **dibujar(Entorno entorno, Princesa princesa)**: dibuja al jefe con la barra de salud superpuesta
- **mover()**: desplaza al jefe e invierte su dirección cuando llega al borde de la isla en la que patrulla
- **rectangulo()**: devuelve el rectángulo de colisión

Clase Mundo

Contenedor central del estado del juego. Almacena todas las entidades activas (princesa, jefe, enemigos, islas, proyectil, castillo, fondo) y provee operaciones de consulta y modificación. Gestiona el arreglo dinámico de enemigos (con redimensionamiento automático) y el arreglo de islas. Los enemigos se mantienen ordenados por ID para permitir búsqueda binaria en `quitarEnemigo()`. Se trata del objeto central al que acceden ``Juego``, ``Isla``, ``Islas``, ``Enemigos``, ``Coordenadas`` y ``Princesa``. Actúa como repositorio compartido de estado de juego.

Propiedades estáticas

- **proporcionAnchoMundo**: la cantidad de veces que entra la pantalla a lo ancho del mundo
- **proporcionAltoMundo**: la cantidad de veces que entra la pantalla a lo alto del mundo

Propiedades de instancia

- **fondo**: el fondo del escenario
- **proyectilPrincesa**: el proyectil disparado por la princesa, puede ser null

- **islas**: un arreglo de islas que se dibujan en pantalla y que no se solapan entre sí
- **largoIslas**: la cantidad de islas que se albergan en el array islas
- **limitesMundo**: un rectángulo que representa al mundo completo
- **limitesPantalla**: un rectángulo que representa la pantalla inicial

Métodos de instancia

- **limitesMundo()**: devuelve el rectángulo con los límites del mundo
- **limitesPantalla(Princesa princesa)**: devuelve el rectángulo con los límites actuales de la pantalla en las coordenadas del mundo
- **fondo()**: devuelve el objeto fondo
- **castillo()**: devuelve el objeto castillo
- **agregarIsla(Isla isla)**: agrega una isla y redimensiona islas si es necesario.
- **islasEnColision(Rectangulo rectangulo, String tipo)**: devuelve una array con las en colisión con el rectángulo provisto
- **enColisionIsla(Rectangulo rectangulo)**: devuelve una array con las en colisión con el rectángulo provisto, pero estas islas colisionan con el rectangulo de manera aumentada, es decir, usar rectángulos más grandes que su tamaño, emulando fronteras espaciales. Esto se usa para que las islas se distribuyan de manera espaciada a la hora de ir incorporándose.
- **borrarIslas()**: elimina todas las islas del mundo
- **islas()**: devuelve un array con las islas del mundo (sin nulos)

Clase Juego

Es la clase principal del juego. Implementa el contrato de InterfaceJuego provisto por la biblioteca entorno.jar. Inicializa todos los objetos del mundo en el constructor y ejecuta el bucle de juego frame a frame en el método tick(). Coordina rendering, detección de colisiones, creación de enemigos y condiciones de victoria/derrota.

Métodos estáticos

- **enColision(Rectangulo r1, Rectangulo r2)**: Devuelve true si los dos rectángulos se superponen
- **tipoDeColision(Rectangulo r1, Rectangulo r2)**: Devuelve una cadena que indica desde qué lado colisionó el primer rectángulo con el segundo: "desde arriba", "desde abajo", "desde la derecha" o "desde la izquierda".
- **transformarX(double x, Princesa princesa, Entorno entorno)**: calcula la coordenada x en pantalla dado un punto del mundo, la posición de la princesa y las dimensiones del entorno.
- **transformarY(double y, Princesa princesa, Entorno entorno)**: calcula la coordenada y en pantalla dado un punto del mundo, la posición de la princesa y las dimensiones del entorno.
- **enteroRandom(int min, int max)**: devuelve un int aleatorio en el intervalo [min, max)
- **cargarYEscalar(String nombreArchivo, double w, double h)**: Carga una imagen y la escala a las dimensiones indicadas, devuelve un objeto tipo java.awt.Image.

Propiedades de instancia

- **entorno**: motor gráfico y de entrada de la biblioteca externa
- **mundo**: estado del juego
- **princesa**: la princesa del juego
- **jefe**: el jefe del juego
- **enemigos**: un array donde se guardan los enemigos
- **largoEnemigos**: la cantidad de enemigos guardados en el array
- **proyectilPrincesa**: el proyectil disparado por la princesa
- **victoria**: indica si la princesa llegó al castillo
- **derrota**: indica si la princesa quedó sin vidas
- **idEnemigoActual**: esta variable privada es el id del último enemigo generado. Es importante porque los enemigos se buscan por id, y deben generarse siempre con ids más grandes que el anterior o se rompería la búsqueda binaria en el array de enemigos.

Métodos de instancia

tick(): especificado por InterfaceJuego dentro de éste método se ejecuta la lógica continua del juego. Se dibuja, se mueven elementos, se detectan colisiones, se generan enemigos y se verifican condiciones de fin de juego.

- **dibujarEnemigos()**: mueve y dibuja cada enemigo activo
- **colisionesYDibujoIslas()**: gestiona colisiones de islas con la princesa y el proyectil, y dibuja las islas
- **colisionesYDibujoJefe()**: detecta colisión del jefe con la princesa (muerte instantánea) y dibuja al jefe
- **colisionesYDibujoProyectilPrincesa()**: gestiona colisiones del proyectil con el jefe y los enemigos
- **colisionesYDibujoPrincesa()**: detecta colisiones de la princesa con enemigos, comprueba límites y la mueve
- **colisionesYDibujoCastillo()**: verifica si la princesa tocó el castillo (victoria) y dibuja el castillo
- **princesaCaeAlVacio()**: resta una vida y reaparece a la princesa en la isla más cercana
- **main(String[] args)**: punto de entrada del programa, instancia Juego que en su constructor llama al método **iniciar()** de entorno
- **agregarEnemigo(Enemigo enemigo)**: agrega un enemigo y redimensiona enemigos si es necesario.
- **quitarEnemigo(Enemigo enemigo)**: busca al enemigo con búsqueda binaria y lo elimina del arreglo
- **indiceDeEnemigo(Enemigo enemigo)**: busca a un enemigo en el array de enemigos basandose en su valor de id (que es un entero) y devuelve su indice.
- **compararEnemigos(Enemigo enemigo1, Enemigo enemigo2)**: compara a dos enemigos entre sí, devuelve 1 si el id de enemigo1 es mayor al de enemigo2, 0 si son iguales, -1 en otro caso.
- **enemigosEnColision(Rectangulo rectangulo)**: devuelve los enemigos que colisionan con el rectangulo provisto

- **establecerProyectilPrincesa(ProyectilPrincesa proyectilPrincesa):**
establece o elimina el proyectil
- **faltanEnemigos():** devuelve true si hay menos enemigos activos que minimoEnemigos

Flujo general del método tick()

- Dibujar fondo
- Si hay victoria o derrota mostrar mensaje, congelar princesa, retornar
- Crear enemigos si faltan
- Purgar entidades fuera de pantalla o muertas
- Procesar disparo de la princesa (clic izquierdo)
- Llamar a colisionesIslas()
- Llamar a colisionesJefe()
- Llamar a colisionesProyectilPrincesa()
- Llamar a dibujarEnemigos()
- Llamar a colisionesCastillo()
- Llamar a colisionesPrincesa()

Conflictos durante el desarrollo del juego

Creación de enemigos

Durante el desarrollo de la clase Enemigos, surgió una dificultad importante. En una primera versión del proyecto los enemigos fueron creados individualmente mediante variables independientes, sin almacenarse en una estructura común. Esta decisión complicaba la detección de colisiones, la eliminación de enemigos y el control de la cantidad mínima de enemigos presentes en pantalla, ya que cada operación debía realizarse sobre cada objeto de forma separada.

Para solucionar este problema se decidió almacenar los enemigos dentro de un arreglo, administrado por la clase Mundo. Gracias a esa modificación fue posible recorrer todos los enemigos mediante iteradores, verificar colisiones de forma uniforme y agregar o eliminar enemigos dinámicamente durante la ejecución del juego sin tener que manejar variables individuales.

El siguiente método fue posible pensarlo porque pasamos de tener enemigos individuales a tener una colección de enemigos administrados por Mundo. Lo que hace este método es verificar si el arreglo está lleno y si es así, crea un arreglo más grande, copia los enemigos del arreglo viejo al nuevo y reemplaza el array viejo.

```
public void agregarEnemigo(Enemigo enemigo) {
    if (largoEnemigos == enemigos.length) {
        Enemigo[] nuevoArray = new Enemigo[enemigos.length * 2];
        System.arraycopy(enemigos, 0, nuevoArray, 0, enemigos.length);
        enemigos = nuevoArray;
    }
}
```

```

    enemigos[largoEnemigos++] = enemigo;
}

```

Además, la forma de generar enemigos es tal que sigue la siguiente formulación matemática, de manera que los enemigos aparezca en lugares donde no colisionen con las islas al desplazarse de manera vertical.

Sea n la cantidad de niveles de islas, e “ y ” las altura posibles los enemigos se generaran en alturas que siguen esta expresión

$$y = (2 * q + 1) * (y_{max} - y_{min}) / (2 * (n - 1)) + y_{min}$$

Donde:

$$q \in [0, \dots, n - 2)$$

y_{min} : es la altura mínima

y_{max} : es la altura máxima

Creación de islas flotantes

Otro conflicto presentado fue en la creación de las islas flotantes. Cada isla era creada como un objeto independiente (isla1, isla2, isla3 ...), algo muy similar a la creación de enemigos. La lógica para evitar que se superpusieran debía programarse manualmente para cada nueva isla. Por ejemplo la segunda isla debía verificar su posición respecto de la primera, la tercera debía compararse con la primera y la segunda y así sucesivamente. A medida que aumentaba la cantidad de islas, el código se volvía extenso y difícil de mantener, ya que cada nueva isla requería agregar nuevas comparaciones.

Como solución se decidió almacenar las islas dentro de un arreglo, administrado por la clase Mundo y al igual que en la creación de enemigos, fue posible recorrer todas las islas mediante iteradores. De esa forma se facilitó verificar colisiones de forma general, sin necesidad de escribir condiciones específicas para cada objeto. La lógica de creación aleatoria continuó respetando la restricción de que las islas no pueden superponerse, pero ahora dicha validación se realiza recorriendo el conjunto de islas existentes.

Esta solución simplificó considerablemente el código, facilitó la incorporación de nuevas islas y permitió reutilizar los mismos mecanismos de detección de colisiones utilizados por otros elementos del juego. Además, se decidió generar las islas a alturas que siguen una fórmula matemática. Sean “ y ” las alturas posibles de las islas y “ n ” la cantidad de niveles de islas, estas siguen la siguiente expresión

$$y = q * (y_{max} - y_{min}) / (n - 1) + y_{min}$$

Donde

$$q \in [0, n)$$

y_{min} : es la altura mínima

y_{max} : es la altura máxima

Reproducción de sonido

Durante la colocación del sonido se presentó un inconveniente relacionado con la reproducción de efectos de sonido. Inicialmente, algunos sonidos fueron ubicados dentro de

condiciones que se evaluaban en cada ejecución del método tick () . Dado que el ciclo principal del juego se ejecuta aproximadamente cada 15 a 30 milisegundos, la condición permanecía verdadera durante varios ciclos consecutivos. Como consecuencia, el mismo archivo de audio intentaba reproducir múltiples veces en un intervalo muy corto de tiempo. Este comportamiento provocó que varias instancias del sonido se superpusieron, generando una reproducción distorsionada, poco clara.

La solución consistió en reproducir los sonidos únicamente cuando ocurría el evento que los generaba ; ejemplo, una colisión o la pérdida de vidas; y no durante cada evaluación de la condición. De esta manera, cada efecto de sonido se ejecuta una sola vez por evento, evitando superposiciones y mejorando la calidad de la experiencia auditiva.

Detección de tipo de colisión

Otro problema fue detectar desde que parte se colisiona un elemento con otro. Se llegó a la conclusión, tal vez no del todo muy eficiente, de que se deben probar primero las 4 condiciones, ya que siempre coexisten al menos 2 de ellas. De ahí se debe determinar cuál es la que domina la naturaleza de la colisión. Esto se logra determinando los segmentos de las aristas en colisión y viendo cual es el más largo, si es el vertical se trata de una colisión de naturaleza horizontal, si es el horizontal se trata de una colisión de naturaleza vertical. Se implementó esta lógica de la siguiente manera

```
public static String tipoDeColision(Rectangulo r1, Rectangulo r2) {
    final double x1 = r1.x();
    final double x2 = r2.x();
    final double y1 = r1.y();
    final double y2 = r2.y();
    final double bordeIzquierdo1 = r1.bordeIzquierdo();
    final double bordeDerecho1 = r1.bordeDerecho();
    final double bordeSuperior1 = r1.bordeSuperior();
    final double bordeInferior1 = r1.bordeInferior();
    final double bordeIzquierdo2 = r2.bordeIzquierdo();
    final double bordeDerecho2 = r2.bordeDerecho();
    final double bordeSuperior2 = r2.bordeSuperior();
    final double bordeInferior2 = r2.bordeInferior();

    double deltaY = 0.0;
    double deltaX = 0.0;
    boolean desdeArriba = bordeInferior1 >= bordeSuperior2 && y1 <= y2;
    boolean desdeAbajo = bordeSuperior1 <= bordeInferior2 && y1 >= y2;
    boolean desdeLaDerecha = bordeDerecho1 >= bordeIzquierdo2 && x1 <= x2;
    boolean desdeLaIzquierda = bordeIzquierdo1 <= bordeDerecho2 && x1 >= x2;

    if (desdeArriba) {
        deltaY = Math.min(bordeInferior1 - bordeSuperior2, r1.alto());
    }

    if (desdeAbajo) {
        deltaY = Math.min(bordeInferior2 - bordeSuperior1, r1.alto());
    }

    if (desdeLaDerecha) {
```

```

        deltaX = Math.min(bordeDerecho1 - bordeIzquierdo2, r1.ancho());
    }

    if (desdeLaIzquierda) {
        deltaX = Math.min(bordeDerecho2 - bordeIzquierdo1, r1.ancho());
    }

    if (deltaY >= deltaX) { // lateral
        if (desdeLaDerecha) {
            return "desde la derecha";
        } else if (desdeLaIzquierda) {
            return "desde la izquierda";
        }
    } else { // vertical
        if (desdeArriba) {
            return "desde arriba";
        } else if (desdeAbajo) {
            return "desde abajo";
        }
    }

    throw new IllegalArgumentException("no hay colisión");
}

```

Conclusión

Durante el desarrollo del proyecto, se implementaron los conocimientos adquiridos durante la cursada; la creación de objetos, métodos y la organización de la lógica en múltiples clases con responsabilidades específicas. Además, nos permitió comprender cómo distintos objetos pueden colaborar entre sí para construir un sistema más complejo.

La clase Juego actúa como punto de inicio y coordina la ejecución general mediante el método tick (), mientras que la clase Mundo administra los distintos elementos que forman parte de la partida, controla su almacenamiento, eliminación y las consultas necesarias para detectar colisiones.

Las entidades principales del juego, como princesa , jefe, isla , enemigo poseen su propio comportamiento, movimiento y representación gráfica. Asimismo, la utilización de las clases auxiliares permiten centralizar tareas específicas, favoreciendo a una mejor organización del programa.

La utilización de rectángulos para representar áreas de colisión permitió implementar un sistema uniforme de interacción entre los distintos objetos del juego. Del mismo modo , el empleo de iteradores, generadores de identificadores y clases de creación contribuyó a mantener una estructura ordenada.

Como resultado se obtuvo un juego funcional en el que la protagonista puede desplazarse, saltar, interactuar con plataformas, enfrentarse a enemigos y utilizar proyectiles gracias a la colaboración entre las distintas clases que componen el sistema.

Implementación

Clase Rectángulo

```
package juego;
import entorno.Entorno;
import java.awt.*;
/**
 * Clase para trabajar con colisiones.
 *
 * @author Miguel Angel Luna Lobos
 */
public class Rectangulo {
    private final double x;
    private final double y;
    private final double ancho;
    private final double alto;
    public Rectangulo(double x, double y, double ancho, double alto) {
        super();
        this.x = x;
        this.y = y;
        this.ancho = ancho;
        this.alto = alto;
    }
    public double x() { return x; }
    public double y() { return y; }
    public double ancho() { return ancho; }
    public double alto() { return alto; }
    /**
     * @return la coordenada y del borde superior (y - alto / 2)
     */
    double bordeSuperior() { return y() - alto() / 2.0; }
    /**
     * @return la coordenada y del borde inferior (y + alto / 2)
     */
}
```



```

double bordeInferior() { return y() + alto() / 2.0; }
/**
 * @return la coordenada x del borde derecho (x + ancho / 2)
 */
double bordeDerecho() { return x() + ancho() / 2.0; }
/**
 * @return la coordenada x del borde izquierdo (x - ancho / 2)
 */
double bordeIzquierdo() { return x() - ancho() / 2.0; }
/**
 * Dibuja el rectángulo con un color dado.
 *
 * @param entorno el entorno
 * @param color el color de relleno
 */
void dibujarRectangulo(Entorno entorno, Color color) {
    entorno.dibujarRectangulo(x(), y(), ancho(), alto(), 0.0, color);
}
public Rectangulo escalar(double proporcionX, double proporcionY) {
    return new Rectangulo(x, y, ancho * proporcionX, alto * proporcionY);
}
/**
 * Devuelve el área del rectángulo.
 * @return el área del rectángulo
 */
public double area(){
    return ancho() * alto();
}
}
}

```

Clase Fondo

```

package juego;
import entorno.Entorno;
import java.awt.*;
//import java.time.Instant;
/**
 * Clase creada para el fondo del juego.
 *
 * @author Noelia Barrionuevo
 */
public class Fondo {
    private final double x;
    private final double y;
    private final Image fondo;
    /**
     * Crea un nuevo fondo.
     * @param x la coordenada x
     * @param y la coordenada y
     * @param fondo la imagen del fondo
     */
    public Fondo(double x, double y, Image fondo) {
        this.x = x;
    }
}

```

```

        this.y = y;
        this.fondo = fondo;
    }
    /**
     * Dibuja el fondo.
     * @param entorno el entorno
     */
    public void dibujar(Entorno entorno) { entorno.dibujarImagen(fondo, x, y, 0); }
}

```

Clase Isla

```

package juego;

import entorno.Entorno;

import java.awt.*;

/**
 * Representa una isla flotante del juego.
 *
 * @author Noelia Barrionuevo
 * @author Miguel Angel Luna Lobos
 */
public class Isla {
    public static double proporcionAlto = 0.8;
    public static double proporcionAlturaMinima = 0.0;
    public static double altoIsla = 20.0;
    public static double anchoMinimo = 400.0;
    public static double anchoMaximo = 500.0;
    public static double factorFronteraAncho = 2.0;
    public static double factorFronteraAlto = 1.0;
    public static int niveles = 8;
    public static double proporcionNivelesAltos = 0.6;
    private static double anchoIslaNivelBajo = 200.0;
    public static double proporcionIslasBajas = 10.0 / (800.0 * 600.0);
    public static double proporcionIslasAltas = 10.0 / (800.0 * 600.0);

    private static Image crearImagenIsla(double ancho) {
        return Juego.cargarYEscalar("isla.png", ancho, altoIsla);
    }

    static double decimalRandom(double min, double max) {
        if (min > max) {
            throw new IllegalArgumentException("max debe ser mayor a min");
        }
        double rango = max - min;
        return Juego.random.nextDouble() * rango + min;
    }

    public static Isla nuevaNivelBajo(Mundo mundo) {
        Isla isla = islaAleatoriaNivelBajo(mundo);
        int intentos = 0;
    }
}

```

```

        while (mundo.islasEnColision(isla.rectangulo(), "fronteraIsla").length > 0) {
            isla = islaAleatoriaNivelBajo(mundo);
            if (intentos++ == 1000) {
                System.out.println("advertencia, no se pudo generar una isla de nivel
bajo");
                return null;
            }
        }
        return isla;
    }
}

```

```

public static Isla nuevaNivelAlto(Mundo mundo) {
    Isla isla = islaAleatoriaNivelAlto(mundo);
    int intentos = 0;
    while (mundo.islasEnColision(isla.rectangulo(), "fronteraIsla").length > 0) {
        isla = islaAleatoriaNivelAlto(mundo);
        if (intentos++ == 1000) {
            System.out.println("advertencia, no se pudo generar una isla de nivel
alto");
            return null;
        }
    }
    return isla;
}
}

```

```

private static Isla islaAleatoriaNivelBajo(Mundo mundo) {
    double anchoMundo = mundo.limiteMundo().ancho();
    double altoMundo = mundo.limiteMundo().alto();
    double alto = proporcionAlto * altoMundo;
    double alturaMinima = proporcionAlturaMinima * altoMundo;
    double yMax = altoMundo - alturaMinima;
    double yMin = altoMundo - alturaMinima - alto;
    int indice = Juego.enteroRandom((int) Math.floor(niveles *
proporcionNivelesAltos), niveles);
    double rango = yMax - yMin;
    double y = indice * rango / (niveles - 1) + yMin;
    double ancho = anchoIslaNivelBajo;
    double xMin = 0.0 + ancho / 2.0;
    double xMax = anchoMundo - ancho / 2.0;
    double x = decimalRandom(xMin, xMax);
    return new Isla(x, y, anchoIslaNivelBajo, altoIsla,
crearImagenIsla(anchoIslaNivelBajo));
}
}

```

```

private static Isla islaAleatoriaNivelAlto(Mundo mundo) {
    double anchoMundo = mundo.limiteMundo().ancho();
    double altoMundo = mundo.limiteMundo().alto();
    double alto = proporcionAlto * altoMundo;
    double alturaMinima = proporcionAlturaMinima * altoMundo;
    double yMax = altoMundo - alturaMinima;
    double yMin = altoMundo - alturaMinima - alto;
    int indice = Juego.enteroRandom(0, (int) Math.floor(niveles *
proporcionNivelesAltos));
}
}

```

```

        double rango = yMax - yMin;
        double y = indice * rango / (niveles - 1) + yMin;
        double ancho = decimalRandom(anchoMinimo, anchoMaximo);
        double xMin = 0.0 + ancho / 2.0;
        double xMax = anchoMundo - ancho / 2.0;
        double x = decimalRandom(xMin, xMax);
        return new Isla(x, y, ancho, altoIsla, crearImagenIsla(ancho));
    }

    private double x;
    private double y;
    private double ancho;
    private double alto;
    private Image isla;

    public Isla(double x, double y, double ancho, double alto, Image isla) {
        this.x = x;
        this.y = y;
        this.ancho = ancho;
        this.alto = alto;
        this.isla = isla;
    }

    /**
     * La coordenada x.
     *
     * @return la coordenada x
     */
    public double x() {
        return x;
    }

    /**
     * La coordenada y.
     *
     * @return la coordenada y
     */
    public double y() {
        return y;
    }

    /**
     * Dibuja la isla.
     *
     * @param entorno el entorno
     * @param princesa la princesa
     */
    public void dibujar(Entorno entorno, Princesa princesa) {
        double x = Juego.transformarX(this.x, princesa, entorno);
        double y = Juego.transformarY(this.y, princesa, entorno);
        entorno.dibujarImagen(isla, x, y, 0);
    }

```

```

/**
 * El rectángulo de colisión de la isla.
 *
 * @return el rectángulo de colisión de la isla
 */
public Rectangulo rectangulo() {
    return new Rectangulo(x, y, ancho, alto);
}
}

```

Clase Castillo

```

package juego;

import entorno.Entorno;

import java.awt.*;

/**
 * Clase del castillo.
 *
 * @author Miguel Angel Luna Lobos
 */
public class Castillo {
    private double x;
    private double y;
    private double ancho;
    private double alto;
    private Image castillo;

    /**
     * Crea un nuevo castillo.
     *
     * @param x          la coordenada x
     * @param y          la coordenada y
     * @param ancho      el ancho
     * @param alto       el alto
     * @param castillo la imagen del castillo
     */
    public Castillo(double x, double y, double ancho, double alto, Image castillo) {
        this.x = x;
        this.y = y;
        this.ancho = ancho;
        this.alto = alto;
        this.castillo = castillo;
    }

    /**
     * Dibuja el castillo.
     *

```

```

    * @param entorno el entorno
    * @param princesa la princesa
    */
    public void dibujar(Entorno entorno, Princesa princesa) {
        double x = Juego.transformarX(this.x, princesa, entorno);
        double y = Juego.transformarY(this.y, princesa, entorno);
        entorno.dibujarImagen(castillo, x, y, 0);
    }

    /**
     * Traslada el castillo a nuevo punto x, y.
     *
     * @param x la coordenada x
     * @param y la coordenada y
     */
    public void trasladar(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double alto() {
        return alto;
    }

    public Rectangulo rectangulo() {
        return new Rectangulo(x, y, ancho * 0.2, alto);
    }
}

```

Clase ProyectilPrincesa

```
package juego;
```

```
import entorno.Entorno;
```

```
import java.awt.*;
```

```

/**
 * Proyectil disparado por la princesa en dirección al cursor del mouse.
 * Se mueve en línea recta a velocidad constante y rebota contra la tierra firme.
 *
 * @author Miguel Angel Luna Lobos
 */
public class ProyectilPrincesa {
    private double x;
    private double y;
    private double ancho;
    private double alto;
    private double sen;
    private double cos;
}

```

```

private Image proyectil;
private double velocidad;

/**
 * Crea un proyectil en la posición indicada con la dirección definida por las
 * componentes trigonométricas del ángulo de disparo.
 *
 * @param x coordenada x inicial
 * @param y coordenada y inicial
 * @param cos coseno del ángulo de disparo (componente horizontal)
 * @param sen seno del ángulo de disparo (componente vertical)
 */
public ProyectilPrincesa(double x, double y, double cos, double sen) {
    this.x = x;
    this.y = y;
    this.sen = sen;
    this.cos = cos;
    this.ancho = 15.0;
    this.alto = 15.0;
    this.velocidad = 8.0;
    this.proyectil = Juego.cargarYEscalar("proyectil_princesa.png", ancho, alto);
}

/**
 * Dibuja el proyectil.
 *
 * @param entorno el entorno
 * @param princesa la princesa
 */
public void dibujar(Entorno entorno, Princesa princesa) {
    double x = Juego.transformarX(this.x, princesa, entorno);
    double y = Juego.transformarY(this.y, princesa, entorno);
    entorno.dibujarImagen(proyectil, x, y, 0);
}

/**
 * Ejecuto el movimiento del proyectil.
 */
public void mover() {
    x += velocidad * cos;
    y += velocidad * sen;
}

public void reboteVertical() {
    sen = -sen;
}

public void reboteHorizontal() {

```

```

        COS = -COS;
    }

    /**
     * El rectángulo de colisión.
     *
     * @return el rectángulo de colisión
     */
    public Rectangulo rectangulo() {
        return new Rectangulo(x, y, ancho, alto);
    }
}

```

Clase Enemigo

```

package juego;

import entorno.Entorno;

import java.awt.*;

/**
 * Representa a un enemigo del juego.
 *
 * @author Noelia Barrionuevo
 * @author Miguel Angel Luna Lobos
 */
public class Enemigo {

    private static double altoEnemigo = 40.0;
    private static double anchoEnemigo = 40.0;

    public static int minimoEnemigos = Isla.niveles / 2;

    private static Image imagenEnemigo = Juego.cargarYEscalar("enemigo.png",
anchoEnemigo, altoEnemigo);

    /**
     * Crea un nuevo enemigo que ingresa en pantalla por la derecha.
     *
     * @param id        el identificador único del enemigo
     * @param mundo     el mundo
     * @param entorno   el entorno
     * @return un nuevo enemigo por derecha
     */
    public static Enemigo nuevoEnemigoDerecha(int id, Princesa princesa, Mundo mundo,
Entorno entorno, Juego juego) {

```



```

        Enemigo enemigo = enemigoAleatorio(id, mundo, princesa.x() + entorno.anch() /
2.0, Math.PI);
        int intentos = 0;
        while (juego.enemigosEnColision(enemigo.rectangulo()).length > 0) {
            enemigo = enemigoAleatorio(id, mundo, princesa.x() + entorno.anch() / 2.0,
Math.PI);
            if (intentos++ == 100) {
                System.out.println("advertencia, no se pudo generar un enemigo");
                return null;
            }
        }
        return enemigo;
    }

    /**
     * Crea un nuevo enemigo que ingresa en pantalla por la derecha.
     *
     * @param id        el identificador único del enemigo
     * @param mundo     el mundo
     * @param entorno   el entorno
     * @return un nuevo enemigo por izquierda
     */
    public static Enemigo nuevoEnemigoIzquierda(int id, Princesa princesa, Mundo mundo,
Entorno entorno, Juego juego) {
        Enemigo enemigo = enemigoAleatorio(id, mundo, princesa.x() - entorno.anch() /
2.0, 0.0);
        int intentos = 0;
        while (juego.enemigosEnColision(enemigo.rectangulo()).length > 0) {
            enemigo = enemigoAleatorio(id, mundo, princesa.x() - entorno.anch() / 2.0,
0.0);
            if (intentos++ == 100) {
                System.out.println("advertencia, no se pudo generar un enemigo");
                return null;
            }
        }
        return enemigo;
    }

    private static Enemigo enemigoAleatorio(int id, Mundo mundo, double x, double angulo)
{
        double altoMundo = mundo.limiteMundo().alto();
        double alto = Isla.proporcionAlto * altoMundo;
        double alturaMinima = Isla.proporcionAlturaMinima * altoMundo;
        double yMax = altoMundo - alturaMinima;
        double yMin = altoMundo - alturaMinima - alto;
        double y = alturaAleatoria(Isla.niveles, yMin, yMax);

        return new Enemigo(id, x, y, anchoEnemigo, altoEnemigo, angulo, imagenEnemigo);
    }

    private static double alturaAleatoria(int n, double yMin, double yMax) {
        if (n < 2) {
            throw new IllegalArgumentException("n no puede ser inferior a 2");
        }
    }

```

```

    }
    int q = Juego.enteroRandom(0, n - 1);
    int indice = 2 * q + 1;
    double rango = yMax - yMin;
    return indice * rango / (2 * (n - 1)) + yMin;
}

```

```

private double x;
private double y;
private double ancho;
private double alto;
private Image enemigo;
private int id;
private double velocidad;
private double angulo;
private boolean vivo = true;

```

```

    Enemigo(int id, double x, double y, double ancho, double alto, double angulo, Image
enemigo) {

```

```

    this.x = x;
    this.y = y;
    this.ancho = ancho;
    this.alto = alto;
    this.enemigo = enemigo;
    this.id = id;
    velocidad = 1.0;
    this.angulo = angulo;
}

```

```

/**
 * El identificador del enemigo.
 *
 * @return el identificador del enemigo
 */
public int id() {
    return id;
}

```

```

/**
 * Dibuja al enemigo.
 *
 * @param entorno el entorno
 * @param princesa la princesa
 */
public void dibujar(Entorno entorno, Princesa princesa) {
    double x = Juego.transformarX(this.x, princesa, entorno);
    double y = Juego.transformarY(this.y, princesa, entorno);
    entorno.dibujarImagen(enemigo, x, y, 0);
}

```

```

/**
 * Mueve al enemigo.

```

```

    */
    public void mover() {
        x = x + velocidad * Math.cos(angulo);
    }

    public void morir() {
        vivo = false;
    }

    /**
     * Indica si este enemigo debe eliminarse.
     *
     * @return true si debe eliminarse, false de lo contrario
     */
    public boolean debeEliminarse() {
        return !vivo;
    }

    /**
     * El rectángulo de colisión del enemigo.
     *
     * @return el rectángulo de colisión del enemigo
     */
    public Rectangulo rectangulo() {
        return new Rectangulo(x, y, ancho, alto);
    }
}

```

Clase Jefe

```

package juego;

import entorno.Entorno;

import java.awt.*;

/**
 * Representa al jefe.
 *
 * @author Noelia Barrionuevo
 * @author Miguel Angel Luna Lobos
 */
public class Jefe {

    public static double altoJefe = 130;
    public static double anchoJefe = 120;
    private double x;
    private double y;
    private double ancho;
    private double alto;
    private Image jefeHaciaDerecha;
    private Image jefeHaciaIzquierda;
}

```

```

private Image jefe;
private double cos;
private double velocidad;
private int vidas;
private Isla isla;

public Jefe(double x, double y, double ancho, double alto, Image jefeHaciaDerecha,
Image jefeHaciaIzquierda) {
    this.x = x;
    this.y = y;
    this.ancho = ancho;
    this.alto = alto;
    this.jefeHaciaDerecha = jefeHaciaDerecha;
    this.jefeHaciaIzquierda = jefeHaciaIzquierda;
    this.jefe = jefeHaciaIzquierda;
    velocidad = 1.0;
    cos = Math.cos(Math.PI);
    isla = null;
    vidas = 10;
}

public void establecerIsla(Isla isla) {
    this.isla = isla;
    this.x = isla.x();
    this.y = isla.y() - alto / 2.0 - Isla.altoIsla / 2.0;
}

public double x() {
    return x;
}

public double y() {
    return y;
}

public double ancho() {
    return ancho;
}

public double alto() {
    return alto;
}

public int vidas() {
    return vidas;
}

/**
 * Dibuja al jefe.
 *
 * @param entorno el entorno
 * @param princesa la princesa
 */

```

```

    public void dibujar(Entorno entorno, Princesa princesa) {
        double x = Juego.transformarX(this.x, princesa, entorno);
        double y = Juego.transformarY(this.y, princesa, entorno);
        Rectangulo rectanguloRojo = new Rectangulo(x, y - alto / 2.0, 100, 10);
        double proporcion = vidas / 10.0;
        double diferencia = rectanguloRojo.anch() - proporcion * 100;
        Rectangulo rectanguloNegro = new Rectangulo(x - diferencia / 2, y - alto / 2.0,
100 * proporcion, 10);
        rectanguloRojo.dibujarRectangulo(entorno, Color.RED);
        rectanguloNegro.dibujarRectangulo(entorno, Color.BLACK);
        entorno.dibujarImagen(jefe, x, y, 0);
    }

    public void mover() {
        if (x <= isla.rectangulo().bordeIzquierdo() || x >=
isla.rectangulo().bordeDerecho()) {
            cos = -cos;
            if (cos < 0) {
                jefe = jefeHaciaIzquierda;
            } else {
                jefe = jefeHaciaDerecha;
            }
        }
        x += velocidad * cos;
    }

    public void pierdeUnaVida() {
        vidas--;
    }

    /**
     * El rectángulo de colisión.
     *
     * @return el rectángulo de colisión
     */
    public Rectangulo rectangulo() {
        return new Rectangulo(x, y, ancho, alto);
    }
}

```

Clase Princesa

```

package juego;

import entorno.Entorno;
import entorno.Herramientas;

import java.awt.*;
import java.time.Instant;

/**
 * Personaje principal del juego.

```

```

*
* @author Miguel Angel Luna Lobos
*/
public class Princesa {
    public static double altoPrincesa = 100;
    public static double anchoPrincesa = 100;
    public static double ladoCorazon = 30.0;
    private Image princesa;
    private Image corazon;
    private double x;
    private double y;
    private double ancho;
    private double alto;
    private double angulo;
    private double velocidad;
    private double velocidadCaidaLibre;
    private double velocidadSalto;
    private boolean enSalto;
    private double aceleracionGravitatoria;
    private Instant marcaTemporalDeCaida;
    private boolean xNoCrece;
    private boolean xNoDecrece;
    private int vidas;

    /**
     * Crea a la princesa en el centro horizontal de la pantalla e inicia la simulación
     * de caída libre.
     *
     * @param x          la coordenada x
     * @param y          la coordenada y
     * @param ancho      el ancho
     * @param alto       el alto
     * @param princesa   la imagen de la princesa
     * @param corazon    la imagen del corazón
     */
    public Princesa(double x, double y, double ancho, double alto, Image princesa, Image
corazon) {
        this.x = x;
        this.y = y;
        this.alto = alto;
        this.ancho = ancho;
        this.princesa = princesa;
        this.corazon = corazon;
        vidas = 10;
        xNoCrece = false; // booleano que indica que no se puede avanzar hacia la derecha
        xNoDecrece = false; // booleano que indica que no se puede avanzar hacia la
izquierda
        velocidad = 3.0; // la velocidad de movimientos laterales
        velocidadSalto = 7.5; // la velocidad que alcanza la princesa tras saltar
        enSalto = false; // indica si la princesa está en un salto
        angulo = 0.0; // el ángulo en el que se mueve la princesa
        aceleracionGravitatoria = 10.0; // la constante g de este mundo
        velocidadCaidaLibre = 0.0; // la velocidad en caída libre, comienza en cero
    }
}

```

```

        marcaTemporalDeCaida = Instant.now(); // iniciamos con la princesa en caída libre
    }

    public int vidas() {
        return vidas;
    }

    public void trasladar(double x, double y) {
        this.x = x;
        this.y = y;
        marcaTemporalDeCaida = Instant.now();
    }

    /**
     * La coordenada x.
     *
     * @return la coordenada x
     */
    public double x() {
        return x;
    }

    /**
     * La coordenada y.
     *
     * @return la coordenada y
     */
    public double y() {
        return y + alto * 0.05;
    }

    /**
     * El ancho.
     *
     * @return el ancho
     */
    public double ancho() {
        return ancho * 0.45;
    }

    /**
     * El alto.
     *
     * @return el alto
     */
    public double alto() {
        return alto * 0.80;
    }

    public void dibujar(Entorno entorno) {
        double espacio = 5.0;
        for (int i = 0; i < vidas; i++) {

```

```

        double l = ladoCorazon * (i + 0.5) + (i + 1) * 5.0;
        entorno.dibujarImagen(corazon, l, espacio + ladoCorazon / 2.0, 0, 1);
    }
    double dx = entorno.ancho() / 2.0;
    double dy = entorno.alto() / 2.0;
    entorno.dibujarImagen(princesa, dx, dy, 0, 1);
}

/**
 * Ejecuta el movimiento de la princesa.
 *
 * @param entorno el entorno
 */
public void mover(Entorno entorno) {
    if (entorno.estaPresionada(entorno.TECLA_DERECHA)) {
        angulo = 0;
        movimientoLateral();
    }
    if (entorno.estaPresionada(entorno.TECLA_IZQUIERDA)) {
        angulo = Math.PI;
        movimientoLateral();
    }
    if ((entorno.sePresiono('a') || entorno.estaPresionada(entorno.TECLA_ARRIBA)) &&
    aceleracionGravitatoria <= 0.1) {
        enSalto = true;
        cayendo();
    }
    gravedad();
    if (enSalto) {
        movimiento(-Math.PI / 2, velocidadSalto);
    }
}

/**
 * Aplica el movimiento lateral de la princesa, reduciendo la velocidad si está
 * en tierra firme para evitar deslizamiento excesivo.
 */
private void movimientoLateral() {
    if (aceleracionGravitatoria <= 0.1) {
        movimiento(angulo, velocidad * 0.75);
    } else {
        movimiento(angulo, velocidad);
    }
}

/**
 * Desplaza a la princesa en la dirección y velocidad indicadas, sin ninguna
 * condición adicional.
 *
 * @param angulo el ángulo de desplazamiento en radianes
 * @param velocidad la velocidad de desplazamiento en píxeles por frame
 */
private void movimiento(double angulo, double velocidad) {

```



```

    double deltaX = Math.cos(angulo) * velocidad;

    if (deltaX > 0.0 && xNoCrece) {
        deltaX = 0.0;
        xNoCrece = false;
    } else if (deltaX < 0 && xNoDecrece) {
        deltaX = 0.0;
        xNoDecrece = false;
    }

    x += deltaX;
    y += Math.sin(angulo) * velocidad;
}

/**
 * Aplica la simulación de gravedad: acumula velocidad de caída libre con el tiempo
 * y la anula cuando la princesa está en tierra firme.
 *
 */
private void gravedad() {
    if (aceleracionGravitatoria <= 0.1) {
        marcaTemporalDeCaida = Instant.now();
    } else {
        double lapso = Instant.now().toEpochMilli() -
marcaTemporalDeCaida.toEpochMilli();
        velocidadCaidaLibre = aceleracionGravitatoria * lapso / 1000;
        movimiento(Math.PI / 2, velocidadCaidaLibre);
    }
    aceleracionGravitatoria = 10.0;
}

public void pierdeUnaVida() {
    vidas--;
}

public void morir() {
    vidas = 0;
}

public void chocarMuroPorIzquierda() {
    enSalto = false;
    xNoDecrece = true;
}

public void chocarTecho() {
    enSalto = false;
}

public void chocarMuroPorDerecha() {
    enSalto = false;
    xNoCrece = true;
}

```

```

private void cayendo() {
    marcaTemporalDeCaida = Instant.now();
    velocidadCaidaLibre = 0.0;
}

public void enTierraFirme() {
    aceleracionGravitatoria = 0.0;
    enSalto = false;
    marcaTemporalDeCaida = null;
    velocidadCaidaLibre = 0.0;
}

/**
 * Dispara un proyectil.
 *
 * @param entorno el entorno
 * @param juego el juego
 */
public void disparar(Entorno entorno, Juego juego) {
    double mouseX = entorno.mouseX() + x - entorno.ancho() / 2.0;
    double mouseY = entorno.mouseY() + y - entorno.alto() / 2.0;
    double distanciaX = mouseX - x;
    double distanciaY = mouseY - y;
    double distancia = Math.sqrt(Math.pow(distanciaX, 2.0) + Math.pow(distanciaY,
2.0));
    double cos = (distanciaX) / distancia;
    double sin = (distanciaY) / distancia;
    ProyectilPrincesa proyectil = new ProyectilPrincesa(x, y, cos, sin);
    try {
        Herramientas.play("recursos/sonidoPoder.wav");
    } catch (Exception error) {
        System.out.println("No se puede reproducir sonido");
    }
    juego.establecerProyectilPrincesa(proyectil);
}

/**
 * El rectángulo de colisión
 *
 * @return el rectángulo de colisión
 */
public Rectangulo rectangulo() {
    return new Rectangulo(x(), y(), ancho(), alto());
}
}

```

Clase Mundo

```

package juego;

import java.util.Objects;

/**

```

```

* Contiene los objetos que se van dibujando en pantalla.
*
* @author Miguel Angel Luna Lobos
* @author Noelia Barrionuevo
*/
public class Mundo {
    public static double proporcionAnchoMundo = 8.0;
    public static double proporcionAltoMundo = 2.0;
    private Isla[] islas;
    private Castillo castillo;
    private int largoIslas;
    private Rectangulo limitesMundo;
    private Rectangulo limitesPantalla;
    private Fondo fondo;

    /**
     * Crea una nueva instancia de Mundo.
     *
     * @param limitesPantalla el rectángulo que indica los límites de la pantalla
     * @param limitesMundo    el rectángulo que indica los límites del mundo
     * @param fondo            el fondo
     * @param castillo        el castillo
     */
    public Mundo(Rectangulo limitesPantalla, Rectangulo limitesMundo, Fondo fondo,
Castillo castillo) {
        this.limitesPantalla = Objects.requireNonNull(limitesPantalla, "limitesPantalla
no puede ser null");
        this.limitesMundo = Objects.requireNonNull(limitesMundo, "limitesMundo no puede
ser null");
        this.fondo = Objects.requireNonNull(fondo, "fondo no puede ser null");
        this.castillo = Objects.requireNonNull(castillo, "castillo no puede ser null");
        islas = new Isla[10];
        largoIslas = 0;
    }

    /**
     * Los límites del mundo.
     *
     * @return un rectángulo que representa los límites del mundo
     */
    public Rectangulo limitesMundo() {
        return limitesMundo;
    }

    /**
     * Los límites de la pantalla.
     *
     * @return un rectángulo que representa los límites de la pantalla.
     */
    public Rectangulo limitesPantalla(Princesa princesa) {
        return new Rectangulo(princesa.x(), princesa.y(), limitesPantalla.ancho(),
limitesPantalla.alto());
    }
}

```

```

/**
 * El fondo del juego.
 *
 * @return el fondo
 */
public Fondo fondo() {
    return fondo;
}

/**
 * El castillo.
 *
 * @return el castillo
 */
public Castillo castillo() {
    return castillo;
}

/**
 * Devuelve un array con las islas en colisión con el rectángulo provisto.
 *
 * @param rectangulo el rectángulo de colisión
 * @param tipo el tipo de elemento
 * @return un array que lista las islas en colisión con el rectángulo argumento
 */
public Isla[] islasEnColision(Rectangulo rectangulo, String tipo) {
    if (tipo.equals("fronteraIsla")) {
        Rectangulo frontera = rectangulo.escalar(Isla.factorFronteraAncho,
Isla.factorFronteraAlto);
        return enColisionIsla(frontera);
    }
    Isla[] almacenador = new Isla[largoIslas];
    int contador = 0;
    for (int i = 0; i < largoIslas; i++) {
        Isla isla_ = islas[i];
        if (Juego.enColision(rectangulo, isla_.rectangulo())) {
            almacenador[contador++] = isla_;
        }
    }
    Isla[] salida = new Isla[contador];
    System.arraycopy(almacenador, 0, salida, 0, contador);
    return salida;
}

private Isla[] enColisionIsla(Rectangulo rectangulo) {
    Isla[] almacenador = new Isla[largoIslas];
    int contador = 0;
    for (int i = 0; i < largoIslas; i++) {
        Isla isla = islas[i];
        Rectangulo frontera = isla.rectangulo().escalar(Isla.factorFronteraAncho,
Isla.factorFronteraAlto);
        if (Juego.enColision(rectangulo, frontera)) {

```

```

        almacenador[contador++] = isla;
    }
}
Isla[] salida = new Isla[contador];
System.arraycopy(almacenador, 0, salida, 0, contador);
return salida;
}

/**
 * Agrega una isla
 *
 * @param isla la isla agregar
 */
public void agregarIsla(Isla isla) {
    if (largoIslas == islas.length) {
        Isla[] nuevoArray = new Isla[islas.length * 2];
        System.arraycopy(islas, 0, nuevoArray, 0, islas.length);
        islas = nuevoArray;
    }
    islas[largoIslas++] = isla;
}

public void borrarIslas() {
    islas = new Isla[10];
    largoIslas = 0;
}

public Isla[] islas() {
    Isla[] copiaIslas = new Isla[largoIslas];
    System.arraycopy(islas, 0, copiaIslas, 0, largoIslas);
    return copiaIslas;
}
}

```

Clase Juego

```

package juego;

import entorno.Entorno;
import entorno.Herramientas;
import entorno.InterfaceJuego;

import java.awt.*;
import java.util.Objects;
import java.util.Random;

public class Juego extends InterfaceJuego {
    public static Random random = new Random();

    static int enteroRandom(int min, int max) {
        if (min > max) {
            throw new IllegalArgumentException("max debe ser mayor a min");
        }
    }
}

```

```

    }
    int rango = max - min;
    // max - min - 1 + min = max - 1
    if (rango == 0) {
        return min;
    }
    return random.nextInt(rango) + min;
}

public static Image cargarYEscalar(String nombreArchivo, double ancho, double alto) {
    Image imagen = Herramientas.cargarImagen("recursos/" + nombreArchivo);
    return imagen.getScaledInstance((int) ancho, (int) alto, Image.SCALE_DEFAULT);
}

/**
 * Determina la dirección desde la que r1 llega a colisionar con r2.
 *
 * @param r1 el rectángulo cuya dirección de llegada se determina
 * @param r2 el rectángulo impactado
 * @return "desde arriba", "desde abajo", "desde la izquierda"
 * o "desde la derecha"
 * @throws IllegalArgumentException si los rectángulos no están en colisión
 */
public static String tipoDeColision(Rectangulo r1, Rectangulo r2) {
    double x1 = r1.x();
    double x2 = r2.x();
    double y1 = r1.y();
    double y2 = r2.y();
    double bordeIzquierdo1 = r1.bordeIzquierdo();
    double bordeDerecho1 = r1.bordeDerecho();
    double bordeSuperior1 = r1.bordeSuperior();
    double bordeInferior1 = r1.bordeInferior();
    double bordeIzquierdo2 = r2.bordeIzquierdo();
    double bordeDerecho2 = r2.bordeDerecho();
    double bordeSuperior2 = r2.bordeSuperior();
    double bordeInferior2 = r2.bordeInferior();

    double deltaY = 0.0;
    double deltaX = 0.0;
    boolean desdeArriba = bordeInferior1 >= bordeSuperior2 && y1 <= y2;
    boolean desdeAbajo = bordeSuperior1 <= bordeInferior2 && y1 >= y2;
    boolean desdeLaDerecha = bordeDerecho1 >= bordeIzquierdo2 && x1 <= x2;
    boolean desdeLaIzquierda = bordeIzquierdo1 <= bordeDerecho2 && x1 >= x2;

    if (desdeArriba) {
        deltaY = Math.min(bordeInferior1 - bordeSuperior2, r1.alto());
    }

    if (desdeAbajo) {
        deltaY = Math.min(bordeInferior2 - bordeSuperior1, r1.alto());
    }

    if (desdeLaDerecha) {

```

```

        deltaX = Math.min(bordeDerecho1 - bordeIzquierdo2, r1.ancho());
    }

    if (desdeLaIzquierda) {
        deltaX = Math.min(bordeDerecho2 - bordeIzquierdo1, r1.ancho());
    }

    if (deltaY >= deltaX) { // lateral
        if (desdeLaDerecha) {
            return "desde la derecha";
        } else if (desdeLaIzquierda) {
            return "desde la izquierda";
        }
    } else { // vertical
        if (desdeArriba) {
            return "desde arriba";
        } else if (desdeAbajo) {
            return "desde abajo";
        }
    }

    throw new IllegalArgumentException("no hay colisión");
}

/**
 * Detecta si dos rectángulos se encuentran o no en colisión.
 *
 * @param r1 el primer rectángulo
 * @param r2 el segundo rectángulo
 * @return true si están en colisión o false de lo contrario
 */
public static boolean enColision(Rectangulo r1, Rectangulo r2) {
    double x1 = r1.x();
    double x2 = r2.x();
    double y1 = r1.y();
    double y2 = r2.y();
    double bordeIzquierdo1 = r1.bordeIzquierdo();
    double bordeDerecho1 = r1.bordeDerecho();
    double bordeSuperior1 = r1.bordeSuperior();
    double bordeInferior1 = r1.bordeInferior();
    double bordeIzquierdo2 = r2.bordeIzquierdo();
    double bordeDerecho2 = r2.bordeDerecho();
    double bordeSuperior2 = r2.bordeSuperior();
    double bordeInferior2 = r2.bordeInferior();
    return ((bordeDerecho1 >= bordeIzquierdo2 && x1 <= x2) || (bordeIzquierdo1 <=
bordeDerecho2 && x1 >= x2)) && ((bordeSuperior1 <= bordeInferior2 && y1 >= y2) ||
(bordeInferior1 >= bordeSuperior2 && y1 <= y2));
}

public static double transformarX(double x, Princesa princesa, Entorno entorno) {
    Rectangulo rectanguloPrincesa = princesa.rectangulo();
    double dx = entorno.ancho() / 2.0;
    return x - rectanguloPrincesa.x() + dx;
}

```

```

}

public static double transformarY(double y, Princesa princesa, Entorno entorno) {
    Rectangulo rectanguloPrincesa = princesa.rectangulo();
    double dy = entorno.alto() / 2.0;
    return y - rectanguloPrincesa.y() + dy;
}

// El objeto Entorno que controla el tiempo y otros
private Entorno entorno;

// Variables y métodos propios de cada grupo
// ...
private Mundo mundo;

private Princesa princesa;

private Jefe jefe;

private Enemigo[] enemigos;
private int largoEnemigos;

private ProyectilPrincesa proyectilPrincesa;
private boolean victoria;
private boolean derrota;

private int idEnemigoActual;

Juego() {
    // Inicializa el objeto entorno
    this.entorno = new Entorno(this, "Super Elizabeth Sis", 800, 600);

    // Inicializar lo que haga falta para el juego
    // ...
    victoria = false;
    derrota = false;
    double anchoPantalla = entorno.ancho();
    double altoPantalla = entorno.alto();
    double anchoMundo = anchoPantalla * Mundo.proporcionAnchoMundo;
    double altoMundo = altoPantalla * Mundo.proporcionAltoMundo;
    Rectangulo limitesPantalla = new Rectangulo(anchoPantalla / 2.0, altoPantalla /
2.0, anchoPantalla, altoPantalla);
    Rectangulo limitesMundo = new Rectangulo(anchoMundo / 2.0, altoMundo / 2.0,
anchoMundo, altoMundo);
    Image imagenPrincesa = cargarYEscalar("princesa.png", Princesa.anchoPrincesa,
Princesa.altoPrincesa);
    Image imagenCorazon = cargarYEscalar("corazon.png", Princesa.ladoCorazon,
Princesa.ladoCorazon);
    princesa = new Princesa(0.0, 0.0, Princesa.anchoPrincesa, Princesa.altoPrincesa,
imagenPrincesa, imagenCorazon);
    Image imagenJefeHaciaDerecha = cargarYEscalar("jefe_hacia_derecha.png",
Jefe.anchoJefe, Jefe.altoJefe);

```



```

        Image imagenJefeHaciaIzquierda = cargarYEscalar("jefe_hacia_izquierda.png",
Jefe.anchoSefe, Jefe.altoJefe);
        jefe = new Jefe(0, 0, Jefe.anchoSefe, Jefe.altoJefe, imagenJefeHaciaDerecha,
imagenJefeHaciaIzquierda);
        Image imagenFondo = cargarYEscalar("fondo.png", anchoPantalla, altoPantalla);
        Fondo fondo = new Fondo(anchoPantalla / 2.0, altoPantalla / 2.0, imagenFondo);
        Image imagenCastillo = cargarYEscalar("castillo.png", Isla.anchoSinimo,
Isla.anchoSinimo);
        Castillo castillo = new Castillo(0.0, 0.0, Isla.anchoSinimo, Isla.anchoSinimo,
imagenCastillo);
        mundo = new Mundo(limitePantalla, limiteMundo, fondo, castillo);
        enemigos = new Enemigo[10];
        largoEnemigos = 0;
        double cantidadIslasBajas = Isla.proporcionIslasBajas *
mundo.limiteMundo().area();
        double cantidadIslasAltas = Isla.proporcionIslasAltas *
mundo.limiteMundo().area();
        int contadorExcepciones = 0;

        Isla primeraIsla = null;
        Isla anteUltimaIsla = null;
        Isla ultimaIsla = null;

        while (contadorExcepciones < 1000) {
            try {
                mundo.borrarIslas();
                int contador = 0;
                while (contador < cantidadIslasBajas) {
                    Isla isla = Isla.nuevaNivelBajo(mundo);
                    if (isla == null) {
                        break;
                    }
                    mundo.agregarIsla(isla);
                    contador++;
                }
                contador = 0;
                double xMin = mundo.limiteMundo().ancho();
                double xMax = 0.0;
                double yMin = mundo.limiteMundo().alto();
                while (contador < cantidadIslasAltas) {
                    Isla isla = Isla.nuevaNivelAlto(mundo);

                    if (isla == null) {
                        break;
                    }
                    if (isla.x() < xMin) {
                        xMin = isla.x();
                        primeraIsla = isla;
                    }
                    if (isla.x() > xMax && isla.y() <= yMin) {
                        xMax = isla.x();
                        yMin = isla.y();
                        anteUltimaIsla = ultimaIsla;
                    }
                }
            }
        }
    }
}

```

```

        ultimaIsla = isla;
    }

    mundo.agregarIsla(isla);
    contador++;
}
Objects.requireNonNull(primerIsla, "no se ha encontrado una isla en la
cual colocar a la princesa");
Objects.requireNonNull(anteUltimaIsla, "no se ha encontrado una isla en
la cual colocar al jefe");
Objects.requireNonNull(ultimaIsla, "no se ha encontrado una isla en la
cual colocar el castillo");
break;
} catch (NullPointerException e) {
    contadorExcepciones++;
}
}
Objects.requireNonNull(primerIsla, "no se ha encontrado una isla en la cual
colocar a la princesa");
Objects.requireNonNull(anteUltimaIsla, "no se ha encontrado una isla en la cual
colocar al jefe");
Objects.requireNonNull(ultimaIsla, "no se ha encontrado una isla en la cual
colocar el castillo");

    princesa.trasladar(primerIsla.x(), primerIsla.y() - Isla.altoIsla / 2.0 -
princesa.alto() / 2.0);
    jefe.establecerIsla(anteUltimaIsla);
    mundo.castillo().trasladar(ultimaIsla.x(), ultimaIsla.y() - Isla.altoIsla / 2.0 -
mundo.castillo().alto() / 2.0);
    // Inicia el juego!
    this.entorno.iniciar();
}

```

/**

* Durante el juego, el método tick() será ejecutado en cada instante y
* por lo tanto es el método más importante de esta clase. Aquí se debe
* actualizar el estado interno del juego para simular el paso del tiempo
* (ver el enunciado del TP para mayor detalle).
*/

```

public void tick() {
    // Procesamiento de un instante de tiempo
    // ...
    mundo.fondo().dibujar(entorno);
    if (victoria) {
        dibujarEnemigos();
        colisionesYDibujoIslas();
        colisionesYDibujoJefe();
        colisionesYDibujoProyectilPrincesa();
        colisionesYDibujoCastillo();
        double x = princesa.x();
        double y = princesa.y();
        princesa.trasladar(x, y);
    }
}

```

```

        entorno.cambiarFont("Arial", 72, Color.BLUE);
        entorno.escribirTexto("¡Ganaste!", entorno.ancho() / 2.0 - 160,
entorno.alto() / 2.0);
        return;
    }
    if (derrota) {
        dibujarEnemigos();
        colisionesYDibujoIslas();
        colisionesYDibujoJefe();
        colisionesYDibujoProyectilPrincesa();
        colisionesYDibujoCastillo();
        double x = princesa.x();
        double y = princesa.y();
        princesa.trasladar(x, y);
        entorno.cambiarFont("Arial", 72, Color.RED);
        entorno.escribirTexto("Perdiste", entorno.ancho() / 2.0 - 150, entorno.alto()
/ 2.0);
        return;
    }

    if (faltanEnemigos()) {
        Enemigo enemigo;

        if (random.nextBoolean()) {
            enemigo = Enemigo.nuevoEnemigoDerecha(idEnemigoActual++, princesa, mundo,
entorno, this);
        } else {
            enemigo = Enemigo.nuevoEnemigoIzquierda(idEnemigoActual++, princesa,
mundo, entorno, this);
        }

        if (enemigo != null) {
            agregarEnemigo(enemigo);
        }
    }

    int i = 0;
    while (i < largoEnemigos) {
        Enemigo enemigo = enemigos[i];
        if (!Juego.enColision(enemigo.rectangulo(), mundo.limitePantalla(princesa))
|| enemigo.debeEliminarse()) {
            quitarEnemigo(enemigo);
        } else {
            i++;
        }
    }
    if (jefe != null && jefe.vidas() <= 0) {
        jefe = null;
    }
    if (proyectilPrincesa != null &&
!Juego.enColision(proyectilPrincesa.rectangulo(), mundo.limitePantalla(princesa))) {
        proyectilPrincesa = null;
    }
}

```

```

        if (entorno.sePresionoBoton(entorno.BOTON_IZQUIERDO) && proyectilPrincesa ==
null) {
            princesa.disparar(entorno, this);
        }
        colisionesYDibujoIslas();
        colisionesYDibujoJefe();
        colisionesYDibujoProyectilPrincesa();
        dibujarEnemigos();
        colisionesYDibujoCastillo();
        colisionesYDibujoPrincesa();
    }

    public boolean faltanEnemigos() {
        return largoEnemigos < Enemigo.minimoEnemigos;
    }

    public void agregarEnemigo(Enemigo enemigo) {
        if (largoEnemigos == enemigos.length) {
            Enemigo[] nuevoArray = new Enemigo[enemigos.length * 2];
            System.arraycopy(enemigos, 0, nuevoArray, 0, enemigos.length);
            enemigos = nuevoArray;
        }
        enemigos[largoEnemigos++] = enemigo;
    }

    public void quitarEnemigo(Enemigo enemigo) {
        int indice = indiceDeEnemigo(enemigo);
        if (indice >= 0) {
            for (int i = indice; i < largoEnemigos - 1; i++) {
                enemigos[i] = enemigos[i + 1];
            }
            enemigos[largoEnemigos - 1] = null;
            largoEnemigos--;
        }
    }

    private int indiceDeEnemigo(Enemigo enemigo) {
        int indiceMinimo = 0;
        int indiceMaximo = largoEnemigos - 1;
        while (indiceMinimo <= indiceMaximo) {
            int indiceIntermedio = (indiceMinimo + indiceMaximo) / 2;
            Enemigo elementoIntermedio = enemigos[indiceIntermedio];
            int comparacion = compararEnemigos(elementoIntermedio, enemigo);

            if (comparacion < 0) {
                indiceMinimo = indiceIntermedio + 1;
            } else if (comparacion > 0) {
                indiceMaximo = indiceIntermedio - 1;
            } else {
                return indiceIntermedio; // Encontrado (comparacion == 0)
            }
        }
    }

```

```

        return -(indiceMinimo + 1);
    }

    private int compararEnemigos(Enemigo enemigo1, Enemigo enemigo2) {
        if (enemigo1.id() < enemigo2.id()) {
            return -1;
        } else if (enemigo1.id() == enemigo2.id()) {
            return 0; // esta condición nunca debería ocurrir porque los ids son únicos
        } else {
            return 1;
        }
    }
}

public Enemigo[] enemigosEnColision(Rectangulo rectangulo) {
    Enemigo[] almacenador = new Enemigo[largoEnemigos];
    int contador = 0;
    for (int i = 0; i < largoEnemigos; i++) {
        Enemigo enemigo = enemigos[i];
        if (Juego.enColision(rectangulo, enemigo.rectangulo())) {
            almacenador[contador++] = enemigo;
        }
    }
    Enemigo[] salida = new Enemigo[contador];
    System.arraycopy(almacenador, 0, salida, 0, contador);
    return salida;
}

private void dibujarEnemigos() {
    for (int i = 0; i < largoEnemigos; i++) {
        Enemigo enemigo = enemigos[i];
        enemigo.mover();
        enemigo.dibujar(entorno, princesa);
    }
}

private void colisionesYDibujoIslas() {
    ProyectilPrincesa proyectil = proyectilPrincesa;
    Isla[] islas = mundo.islas();
    for (Isla isla : islas) {
        Rectangulo rectanguloIsla = isla.rectangulo();
        if (enColision(princesa.rectangulo(), rectanguloIsla)) {
            String tipoDeColision = Juego.tipoDeColision(princesa.rectangulo(),
rectanguloIsla);

            switch (tipoDeColision) {
                case "desde arriba":
                    princesa.trasladar(princesa.x(), rectanguloIsla.y() -
rectanguloIsla.alto() / 2.0 - princesa.alto() / 2.0);
                    princesa.enTierraFirme();
                    break;
                case "desde abajo":
                    princesa.chocarTecho();
                    break;
            }
        }
    }
}

```

```

        case "desde la derecha":
            princesa.chocarMuroPorDerecha();
            break;
        case "desde la izquierda":
            princesa.chocarMuroPorIzquierda();
            break;
        default:
            throw new IllegalArgumentException("tipo de colisión " +
tipoDeColision + " no válido");
    }
}
if (proyectil != null && enColision(proyectil.rectangulo(), rectanguloIsla))
{
    String tipoDeColision = Juego.tipoDeColision(proyectil.rectangulo(),
rectanguloIsla);
    switch (tipoDeColision) {
        case "desde arriba":
        case "desde abajo":
            proyectil.reboteVertical();
            break;
        case "desde la derecha":
        case "desde la izquierda":
            proyectil.reboteHorizontal();
            break;
        default:
            throw new IllegalArgumentException("tipo de colisión %s no
soportado".formatted(tipoDeColision));
    }
}
isla.dibujar(entorno, princesa);
}
}

private void colisionesYDibujoJefe() {
    if (jefe != null && jefe.vidas() <= 0) {
        jefe = null;
    }
    if (jefe == null) {
        return;
    }
    if (enColision(jefe.rectangulo(), princesa.rectangulo())) {
        princesa.morir();
    }
    jefe.mover();
    jefe.dibujar(entorno, princesa);
}

private void colisionesYDibujoProyectilPrincesa() {
    if (proyectilPrincesa == null) {
        return;
    }
    if (jefe != null && enColision(jefe.rectangulo(),
proyectilPrincesa.rectangulo())) {

```

```

        proyectilPrincesa = null;
        jefe.pierdeUnaVida();
        // El jefe emite sonido cuando un proyectil lo colisiona
        try {
            Herramientas.play("recursos/dragon.wav");
        } catch (Exception error) {
            System.out.println("No se puede reproducir sonido de dragon");
        }
        return;
    }

    Enemigo[] enemigosEnColision =
    enemigosEnColision(proyectilPrincesa.rectangulo());
    if (enemigosEnColision.length > 0) {
        proyectilPrincesa = null;
        for (Enemigo enemigo : enemigosEnColision) {
            enemigo.morir();
        }
        return;
    }

    proyectilPrincesa.mover();
    proyectilPrincesa.dibujar(entorno, princesa);
}

private void colisionesYDibujoPrincesa() {
    if (princesa.vidas() <= 0) {
        derrota = true;
    }
    if (princesa.y() > mundo.limiteMundo().alto() * 2.5) {
        // se agrega sonido cuando la princesa cae al vacío y pierde una vida
        try {
            Herramientas.play("recursos/dragon2.wav");
        } catch (Exception error) {
            System.out.println("Princesa cae al vacio no funciona el sonido");
        }
        princesaCaeAlVacio();
    }
    Enemigo[] enemigosEnColision = enemigosEnColision(princesa.rectangulo());
    for (Enemigo enemigo : enemigosEnColision) {
        enemigo.morir();
        // la princesa emite sonido cuando le sacan vidas
        try {
            Herramientas.play("recursos/PrincesaPierdeVidas.wav");
        } catch (Exception error) {
            System.out.println("Sonido de pierde vidas con enemigo");
        }
        princesa.pierdeUnaVida();
    }
    princesa.mover(entorno);
    princesa.dibujar(entorno);
}

```

```

private void princesaCaeAlVacio() {
    princesa.pierdeUnaVida();
    if (princesa.vidas() <= 0) {
        derrota = true;
        return;
    }

    Isla islaMasCercana = null;
    double deltaX = Double.POSITIVE_INFINITY;
    Isla[] islas = mundo.islas();
    for (Isla isla : islas) {
        double nuevoDeltaX = Math.abs(isla.x() - princesa.x());
        if (nuevoDeltaX <= deltaX) {
            deltaX = nuevoDeltaX;
            islaMasCercana = isla;
        }
    }

    Objects.requireNonNull(islaMasCercana, "error fatal, no se ha encontrado una isla
donde colocar a la princesa");
    princesa.trasladar(islaMasCercana.x(), islaMasCercana.y() - Isla.altoIsla / 2.0 -
princesa.alto() / 2.0);
}

private void colisionesYDibujoCastillo() {
    Castillo castillo = mundo.castillo();
    if (enColision(princesa.rectangulo(), castillo.rectangulo())) {
        victoria = true;
    }
    mundo.castillo().dibujar(entorno, princesa);
}

@SuppressWarnings("unused")
public static void main(String[] args) {
    Juego juego = new Juego();
}

public void establecerProyectilPrincesa(ProyectilPrincesa proyectil) {
    proyectilPrincesa = proyectil;
}
}

```